

Heuristics for Mathematical Optimization

Project 2026

Domenico Salvagnin

March 30, 2026

The project consists in implementing a primal heuristic for the set covering problem.

Set Covering Problem

Given a ground set I and a family F of subsets F_j of I , each with cost $c_j > 0$, the set covering problem consists in looking for a subset of F that covers all the elements of the ground set with minimum total cost. The problem can be easily given a integer linear programming formulation:

$$\begin{aligned} \min \quad & \sum_{j=1}^n c_j x_j \\ \sum_{j:i \in F_j} x_j & \geq 1 \quad \forall i \in I \\ x_j & \in \{0, 1\} \quad \forall j \in \{1, \dots, n\} \end{aligned}$$

where binary variables x_j encode whether the subset F_j is taken or not in the optimal solution.

Instances

The heuristics will be tested on the *rail** instances in the ORlib:

<http://people.brunel.ac.uk/~mastjjb/jeb/orlib/scpinfo.html>

This consists in a list of seven relatively large instances arising from an application in Italian railways.

Instances are described in *column-major*, with the following format. The first line contains the number of rows (m) and number of columns (n). Then we have a row for each column $j \in \{1, \dots, n\}$: each such row gives the cost of the column, the number of rows that it covers and the corresponding list of row indices (with 1-based indexing).

Rules

You are free to implement the heuristic as you see fit. The code is supposed to be run on a single instance at the time, with a time limit of 10 minutes, and with reasonable hardware resources (4 cores and at most 16GB of RAM).

Your script/executable should be named `sc_solver` (without any extension) and will be run with the following command:

```
./sc\_solver path_to_input_instance 2>&1 >path_to_log_file
```

Your code can produce any output, but it must print lines of the format:

```
#### Feasible solution of value xxx [time yyy]
```

where `xxx` is the solution objective value and `yyy` is the elapsed time in seconds since the start of your method. In addition, for each solution found by your method, you must create a file named `instance.number.sol`, where `number` is a counter increasing from one to the number of solutions found, that contains in the first line the objective value of the solution and in the second line the list of column indices (0-based) of the solution. Those solutions files will be checked by an independent solution checker for correctness.

For each instance, I will use the primal integral to assign a score to your heuristic. Let $\tilde{x}$ be a feasible solution for an instance and x^* be an optimal (or best known) solution for the same instance. The *primal gap* $\gamma \in [0, 1]$ of $\tilde{x}$ is defined as:

$$\gamma(\tilde{x}) = \frac{c^T \tilde{x} - c^T x^*}{c^T \tilde{x}}$$

Let $T \in \mathbb{R}_+$ be a limit on the solution time of your method. The *primal gap function* $p(t) : [0, T] \mapsto [0, 1]$ is defined as 1 if no incumbent is known at time t and as $\gamma(\tilde{x}_t)$ if $\tilde{x}_t$ is the incumbent at time t . Thus, the primal gap function $p(t)$ is a monotonically decreasing step function that changes whenever a new incumbent is found: it is equal to one at $t = 0$, and zero from the point on at which the optimal solution is found. The primal integral $P(T)$ is just the integral of $p(t)$ over the interval $[0, T]$.

The score assigned to a given model, will be:

$$S = 1 - \frac{P(T)}{T}$$

with $T = 600$. Notice that this is equal to zero if no solution is found within the timelimit T and to one if the optimal solution is found straight away at time $t = 0$.

Also, we provide below reference values for the 7 instances in the testbed: the first is obtained by the obvious greedy approach (that you are expected to improve upon) and the second is the best known solution.

instance	greedy	best known
rail507	216	174
rail516	204	182
rail582	251	211
rail2536	894	689
rail2586	1166	960
rail4284	1376	1077
rail4872	1902	1556

Submission

You need to submit the source code of your heuristic (in a language of choice between C/C++/Python), together with a CSV file containing, for each instance, the best primal bound and corresponding time. You can send code and results to me by email.

Deadline: May 31st, 2026.